

Aula 12

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) Low-No-Code - Teoria	3
2) Princípios SOLID - Teoria	8
3) Princípios SOLID - Questões Comentadas	15
4) Princípios SOLID - Lista de Questões	22

DESENVOLVIMENTO LOW/NO CODE

Conceitos Básicos

INCIDÊNCIA EM PROVA: BAIXA

Analogia

Imagine que você precisa construir uma casa, mas não sabe nada sobre construção. O jeito seria contratar um arquiteto para projetar a casa e um construtor para executar o projeto.

No entanto, com o low-code, é como se você pudesse contar com um kit de peças pré-fabricadas, onde você só precisaria encaixar as peças de acordo com o projeto pré-definido. Isso permite que você possa construir uma casa sem precisar aprender todo o processo de construção desde o começo.

Já o no-code é ainda mais simples. É como se você comprasse uma casa já pronta, onde tudo que você precisa fazer é decorar e personalizar o espaço de acordo com as suas necessidades. Você não precisa saber nada sobre construção, basta escolher o modelo da casa que mais lhe agrada e personalizá-la conforme seus gostos e necessidades.

De forma análoga, o low-code e no-code são plataformas de desenvolvimento de software que permitem a criação de aplicativos com pouco ou nenhum conhecimento de programação. Com essas plataformas, é possível criar soluções de software com mais rapidez e eficiência, permitindo que até mesmo pessoas sem formação em programação possam desenvolver seus próprios aplicativos.

Vamos iniciar falando sobre o desenvolvimento de software tradicional – também chamado de **Desenvolvimento High Code**. Esse tipo de desenvolvimento utiliza linguagens de programação completas e frameworks avançados para criar aplicações complexas e altamente personalizáveis. Diferente das abordagens Low Code e No Code, o High Code exige um conhecimento aprofundado em programação, design de software, e arquitetura de sistemas.

O **Desenvolvimento Low Code** (também chamada de Codeless) e **Desenvolvimento No Code** são termos que vêm ganhando cada vez mais espaço no mundo do desenvolvimento de software. Ambos se referem a abordagens que visam tornar a criação de aplicativos mais fácil, rápida e acessível para um público mais amplo, incluindo usuários finais, analistas de negócios e desenvolvedores com pouco conhecimento técnico.

Em termos simples, **Low Code refere-se a uma abordagem de desenvolvimento de software que se concentra em fornecer uma plataforma ou conjunto de ferramentas que permitem que os usuários criem aplicativos com o mínimo de programação possível**. Isso é alcançado por meio de interfaces visuais, assistentes e modelos pré-construídos que podem ser personalizados de acordo com as necessidades do usuário.

Já o No Code é uma abordagem que busca permitir que usuários sem conhecimento técnico possam criar aplicativos por conta própria, sem precisar escrever uma única linha de código. Isso é possível por meio de plataformas que oferecem ferramentas de arrastar e soltar, modelos pré-construídos e automação de processos, permitindo que usuários finais construam aplicativos personalizados para suas necessidades específicas.

Uma das principais vantagens de ambas as abordagens é a redução do tempo de desenvolvimento e a facilitação da criação de aplicativos. **Com menos tempo gasto em codificação, os desenvolvedores podem se concentrar em outros aspectos do processo de desenvolvimento, como testes, implantação e manutenção.**

Outra vantagem é a democratização do desenvolvimento de software. Com plataformas Low Code e No Code, usuários finais e analistas de negócios podem criar aplicativos personalizados que atendam às suas necessidades sem depender de desenvolvedores especializados. Isso pode melhorar a eficiência e a produtividade da organização.

No entanto, uma das desvantagens dessas abordagens é a limitação na personalização e no escopo do aplicativo criado. **Os modelos pré-construídos podem não atender a todas as necessidades do usuário e pode haver limitações na integração com outras ferramentas ou sistemas.** Além disso, a automação de processos pode levar a uma falta de controle sobre o aplicativo e sua funcionalidade.

No fim das contas, podemos afirmar que Low Code e No Code são abordagens de desenvolvimento de software que buscam democratizar e simplificar o processo de criação de aplicativos. Embora ofereçam vantagens significativas, também apresentam limitações em termos de personalização e integração. É importante avaliar cuidadosamente as necessidades do usuário e as funcionalidades necessárias antes de escolher uma abordagem de desenvolvimento.

As plataformas de desenvolvimento Low Code e No Code são baseadas em **ferramentas de desenvolvimento visual** que visam reduzir o tempo de desenvolvimento e o esforço necessário para criar aplicativos. Low Code e No Code são abordagens semelhantes, mas com diferenças significativas.

O Low Code é uma abordagem que utiliza uma **quantidade reduzida de programação tradicional**, onde o desenvolvedor ainda precisa escrever algum código, mas a maior parte da lógica é gerada automaticamente. Já o No Code, como o próprio nome sugere, **não requer que o usuário escreva nenhum código**, apenas utiliza as ferramentas disponíveis para criar aplicativos.

Existem diversas plataformas Low Code e No Code disponíveis no mercado. Algumas das mais populares são: **Microsoft Power Apps, Google App Maker, Salesforce Lightning, OutSystems, Mendix, Bubble, Airtable e Webflow.** Cada uma dessas plataformas tem suas próprias características e funcionalidades, que podem ser escolhidas com base nas necessidades e objetivos do usuário.

Por exemplo, o **Microsoft Power Apps** é uma plataforma Low Code que oferece um ambiente de desenvolvimento visual para criar aplicativos móveis e da web, enquanto o **Bubble** é uma plataforma No Code que permite criar aplicativos complexos sem escrever nenhuma linha de código. O **OutSystems** é uma plataforma Low Code que oferece recursos de automação de negócios para acelerar o desenvolvimento de aplicativos corporativos.

Cada plataforma tem suas próprias vantagens e desvantagens, dependendo do contexto e dos requisitos de negócios. Algumas das vantagens incluem maior produtividade, redução de custos e maior agilidade no desenvolvimento. **No entanto, as desvantagens podem incluir limitações de personalização, maior dependência da plataforma e menor controle sobre o código fonte.**

A integração e extensibilidade são características fundamentais em qualquer plataforma, especialmente nas plataformas Low Code e No Code, que visam facilitar a criação de aplicativos por usuários sem conhecimentos avançados em programação. **Porém, é importante que essas plataformas possam ser integradas com outras ferramentas e sistemas existentes, a fim de expandir suas funcionalidades e atender às necessidades específicas de cada projeto.**

A integração pode ser realizada de diversas maneiras, dependendo da plataforma e das ferramentas envolvidas. **Uma opção é utilizar APIs (Application Programming Interfaces)**, que permitem a comunicação entre diferentes sistemas de forma padronizada e segura. Outra opção é o uso de webhooks, que permitem que a plataforma envie informações para outros sistemas quando ocorrem eventos específicos, como a criação de um novo registro em um banco de dados.

Já a extensibilidade das plataformas permite que os usuários possam adicionar novas funcionalidades ou personalizar a plataforma de acordo com suas necessidades. Isso pode ser feito por meio de plugins, que são módulos adicionais que se integram à plataforma e adicionam novas funcionalidades. Também é possível utilizar linguagens de programação para criar novos componentes e integrá-los à plataforma.

Ao integrar e estender plataformas Low Code e No Code, é possível criar soluções mais complexas e personalizadas, sem a necessidade de investir em desenvolvimento de software tradicional. Além disso, as plataformas Low Code e No Code oferecem uma maior facilidade de uso e uma redução significativa no tempo e nos custos de desenvolvimento. Por isso, é importante que as empresas e organizações considerem essas plataformas em suas estratégias de desenvolvimento de software.

CARACTERÍSTICAS	HIGH CODE	LOW CODE	NO CODE
NÍVEL DE CODIFICAÇÃO	Extensível	Mínimo	Nenhuma
PERSONALIZAÇÃO	Altamente Personalizável	Moderadamente Personalizável	Baixa Personalização

COMPLEXIDADE DO PROJETO	Adequado para projetos complexos	Adequado para projetos de complexidade média	Adequado para projetos simples
TEMPO DE DESENVOLVIMENTO	Longo	Rápido	Muito rápido
CUSTO DE DESENVOLVIMENTO	Alto	Moderado	Baixo
CONHECIMENTO NECESSÁRIO	Alto	Moderado	Baixo
CONTROLE	Total	Limitado	Muito limitado
INTEGRAÇÃO	Avançada	Moderada	Limitada
ESCALABILIDADE	Alta	Moderada	Baixa
USO DE FRAMEWORKS	Sim	Às vezes	Não
AUTOMAÇÃO	Menos automação (mais controle manual)	Alguma automação	Alta automação
ATUALIZAÇÕES E MANUTENÇÃO	Complexa	Simples a moderada	Simples
VELOCIDADE DE INOVAÇÃO	Lenta	Rápida	Muito Rápida
SEGURANÇA	Alta	Moderada	Baixa
EXPERIÊNCIA DE USUÁRIO	Altamente customizada	Boa, mas menos flexível	Limitada pela ferramenta

(PGE/RJ – 2022) Uma das vantagens dos conceitos codeless e nocode sobre o desenvolvimento tradicional é o lançamento mais rápido de protótipos e do MVP (mínimo produto viável) do sistema.

Comentários: uma das principais vantagens das abordagens codeless e nocode é a rapidez no desenvolvimento de protótipos e do MVP (Minimum Viable Product), que é a versão mais simples e funcional do produto. Essas abordagens oferecem uma variedade de ferramentas que permitem que os usuários desenvolvam aplicativos rapidamente, sem a necessidade de escrever uma grande quantidade de código ou contratar desenvolvedores. Além disso, essas ferramentas geralmente possuem recursos pré-construídos, como modelos de interface do usuário e bibliotecas de código, que podem ser usados para acelerar ainda mais o processo de desenvolvimento. Com o MVP lançado mais rapidamente, as equipes de desenvolvimento podem validar a ideia com os usuários mais cedo, testar a funcionalidade e receber feedback para iterar e melhorar o produto de forma mais rápida e eficiente. Isso pode economizar tempo e dinheiro, além de aumentar as chances de sucesso do produto no mercado (Correto).

(TJ/CE – 2022) Considere a lista de aplicações abaixo.

- I. Assistentes virtuais e ferramentas de chatbot
- II. App de internet banking.
- III. Ferramenta RPA para automatizar um processo administrativo de back-office.
- IV. App para divulgação de campanha de e-mail marketing.
- V. Ferramenta para controle automatizado de carro autônomo.

As aplicações que pertencem a categorias típicas de sistemas Low-code/No-Code são as que constam APENAS em:

- a) I, II e V.
- b) II, III e IV.
- c) III e IV.
- d) I, III e IV.
- e) I, IV e V.

Comentários: (I) Correto. Assistentes virtuais e ferramentas de chatbots podem ser criados com plataformas de No-Code, que permitem a criação de fluxos de conversação sem a necessidade de escrever código; (II) Errado, apps de internet banking não são aplicações típicas de desenvolvimento Low/No Code; (III) Correto, ferramentas de RPA (Robotic Process Automation) – tecnologia de automação de processos de negócio – são tipicamente implementadas por meio de desenvolvimento Low/No Code; (IV) Correto. App para divulgação de campanha de e-mail marketing pode ser criado com ferramentas de Low-Code, que possuem recursos pré-construídos para lidar com o envio de e-mails em massa; (V) Errado. Ferramenta para controle automatizado de carro autônomo não são típicas de sistemas Low/No Code (Letra D).

(BNB – 2022) Low code foi criado para desenvolver interfaces que funcionassem sem alfabetização em nenhum idioma e, principalmente, sem alfabetização em inglês.

Comentários: o low code é uma abordagem que visa simplificar o processo de desenvolvimento de aplicativos, permitindo que usuários sem habilidades de programação possam criar aplicativos rapidamente usando ferramentas visuais e recursos pré-construídos, independentemente do idioma utilizado (Errado).

(CEBRASPE / SERPRO – 2023) Low-code e no-code são abordagens que utilizam plataformas visuais e intuitivas para facilitar o desenvolvimento de aplicativos de forma segura, sem oferecer riscos.

Comentários: Low-code e no-code são abordagens que utilizam plataformas visuais e intuitivas para facilitar e acelerar o desenvolvimento de aplicativos, permitindo que desenvolvedores e até usuários não técnicos criem aplicações com pouca ou nenhuma necessidade de codificação. No entanto, essas abordagens não são isentas de riscos. Como qualquer outra tecnologia, elas podem apresentar riscos relacionados à segurança, escalabilidade, e manutenção, especialmente se não forem utilizadas com as melhores práticas ou sem o devido controle e governança (Errado).

PRINCÍPIOS SOLID

Conceitos Básicos

PRINCÍPIOS SOLID

Trata-se de um conjunto de princípios de design criados para tornar o software mais fácil de manter, entender, sustentar, escalar e estender por meio do desenvolvimento de classes fracamente acopladas. Os cinco princípios são: Princípio de Responsabilidade Única, Princípio Aberto-Fechado, Princípio de Substituição de Liskov, Princípio de Segregação de Interface e Princípio de Inversão de Dependência.

Escrever um código que satisfaça os requisitos atuais, e que também possa satisfazer requisitos futuros facilmente, deve ser o objetivo de qualquer desenvolvedor – evoluir com o tempo é o único fator que pode manter o código-fonte. Princípios SOLID são cinco princípios de design de código orientado a objeto para tornar o código mais entendível, claro, flexível, conciso e tolerante a mudanças; e para aumentar a adesão do código aos princípios da orientação a objetos.



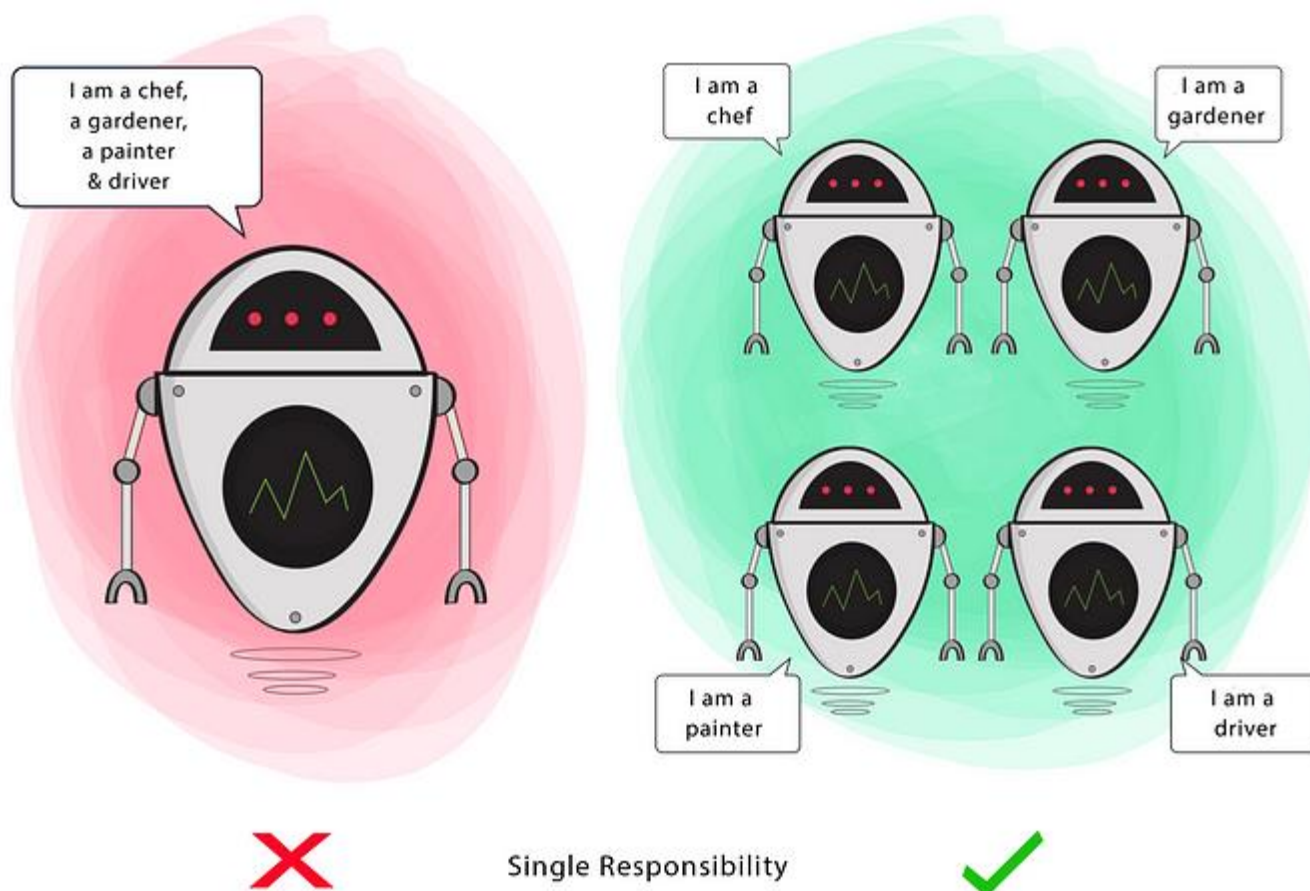
SOLID é um acrônimo para cada um dos cinco princípios que fazem parte desse grupo:

- *Single Responsibility Principle;*
- *Open/Closed Principle*
- *Liskov Substitution Principle*
- *Interface Segregation Principle*
- *Dependency Inversion Principle*

Esses cinco princípios de desenvolvimento de software são diretrizes a serem seguidas ao criar software para facilitar o dimensionamento e a manutenção. Veja bem, alguns desses princípios podem parecer semelhantes, mas não visam o mesmo objetivo. Para simplificar o

acompanhamento, usarei a palavra "Classe", mas observe que ela também pode se aplicar a uma função, método ou módulo.

Responsabilidade Única

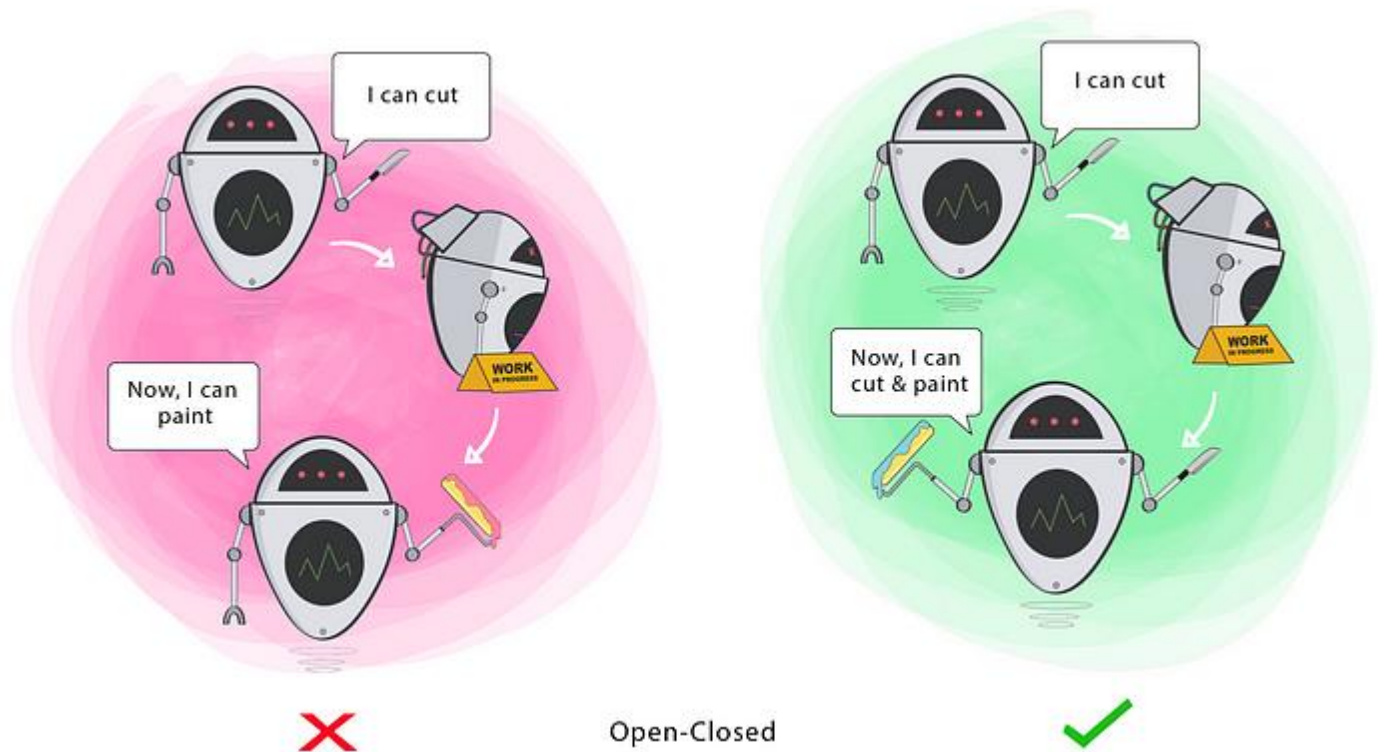


UMA CLASSE DEVER TER UMA ÚNICA RESPONSABILIDADE

Se uma classe tiver muitas responsabilidades, aumenta a probabilidade de ocorrerem bugs, uma vez que alterar uma de suas responsabilidades pode afetar as outras sem que você saiba. Para resolver esse problema, temos o Princípio da Responsabilidade Única. O objetivo desse princípio é separar comportamentos para que, se surgirem bugs como resultado de sua alteração, isso não afete outros comportamentos não relacionados.

Na imagem, temos um robô que tem muitas responsabilidades: ele é chef, jardineiro, pintor e motorista – o ideal é que tivéssemos um robô para cada uma dessas responsabilidades.

Aberto-Fechado

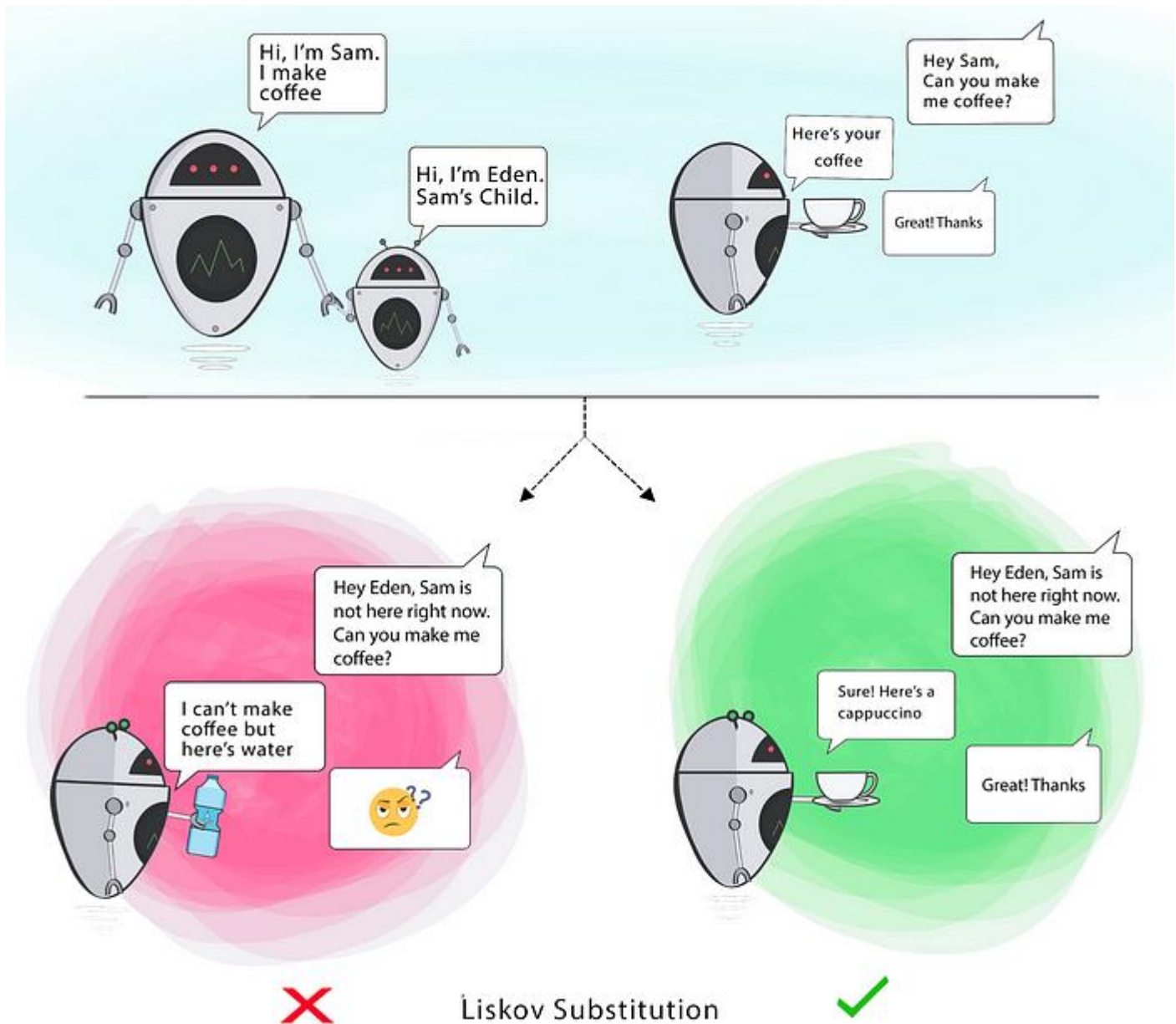


CLASSES DEVEM SER ABERTAS PARA EXTENSÃO, MAS FECHADAS PARA MODIFICAÇÃO

Alterar o comportamento atual de uma classe afetará todos os sistemas que usam essa classe. Se você deseja que a classe execute mais funções, a abordagem ideal é adicionar às funções que já existem e, não, alterá-las. Aberto para extensão significa que, ao receber uma nova requisição, é possível adicionar um novo comportamento. Fechado para modificação significa que, para introduzir um novo comportamento (extensão), não é necessário modificar o código existente.

O objetivo desse princípio é estender o comportamento de uma classe sem alterar o comportamento existente dela. Isso evita a ocorrência de bugs onde quer que a classe esteja sendo usada. Na imagem à esquerda, observem que tínhamos um robô que cortava, teve seu comportamento modificado e que agora passou a pintar; na imagem à direita, tínhamos um robô que cortava, foi estendido e agora aprendeu **também** a pintar.

Substituição de Liskov



SE S É UM SUBTIPO DE T, ENTÃO OBJETOS DO TIPO T EM UM PROGRAMA PODEM SER SUBSTITUÍDOS POR OBJETOS DO TIPO S SEM ALTERAR AS PROPRIEDADES DESEJÁVEIS DESSE PROGRAMA

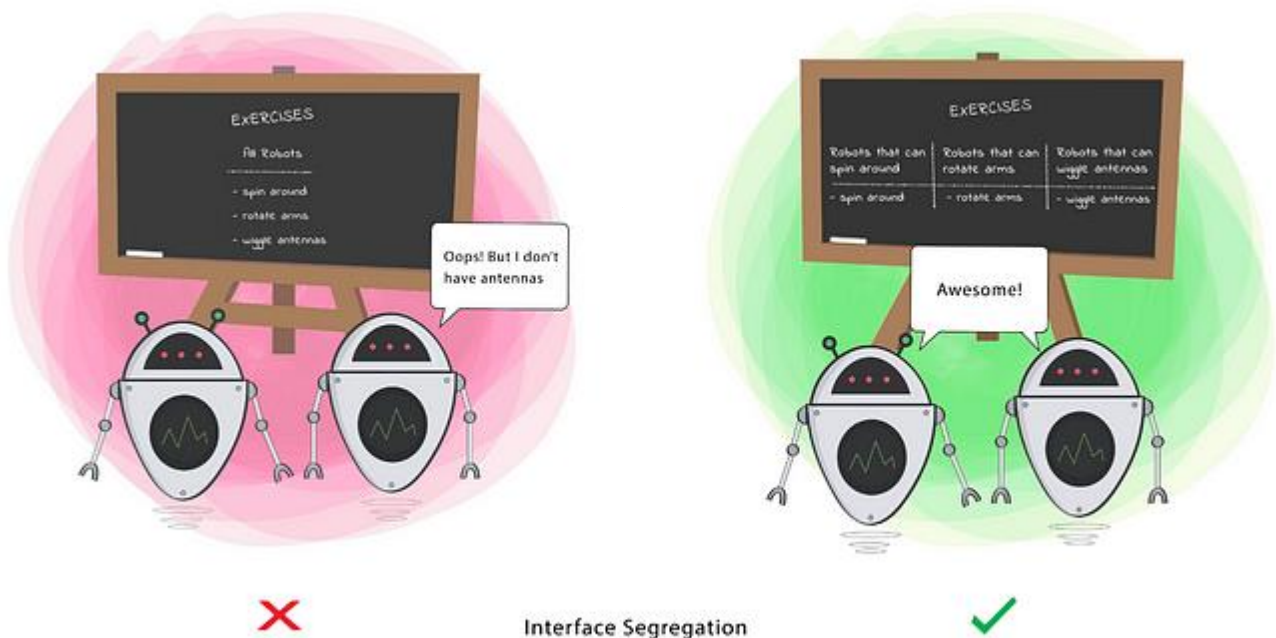
Quando uma classe-filha não pode executar as mesmas ações que sua classe-pai, isso pode causar bugs. Se você tiver uma classe e criar outra classe a partir dela, ela se tornará pai e a nova classe se tornará filha. A classe-filha deve ser capaz de fazer tudo o que a classe-pai pode fazer. Este processo é chamado de Herança. A classe-filha deve ser capaz de processar as mesmas solicitações e entregar o mesmo resultado que a classe-pai ou pode entregar um resultado do mesmo tipo.

O objetivo desse princípio é reforçar a consistência para que a classe-pai ou sua classe-filha possam ser usadas da mesma maneira sem erros. A imagem à esquerda mostra que a classe-pai entrega café (pode ser qualquer tipo de café). É aceitável que a classe-filha entregue *cappuccino* por ser um

tipo específico de café, mas não é aceitável entregar água. Se a classe-filha não atender a esses requisitos, isso significa que a classe-filha foi alterada completamente e viola esse princípio.

Já a imagem à direita mostra que Eden (que é filho de Sam) também faz café (*cappuccino*), já que ele pode executar as mesmas ações que sua classe-pai (Sam).

Segregação de Interface



CLIENTES NÃO DEVEM SER FORÇADOS A DEPENDER DE MÉTODOS QUE NÃO UTILIZAM

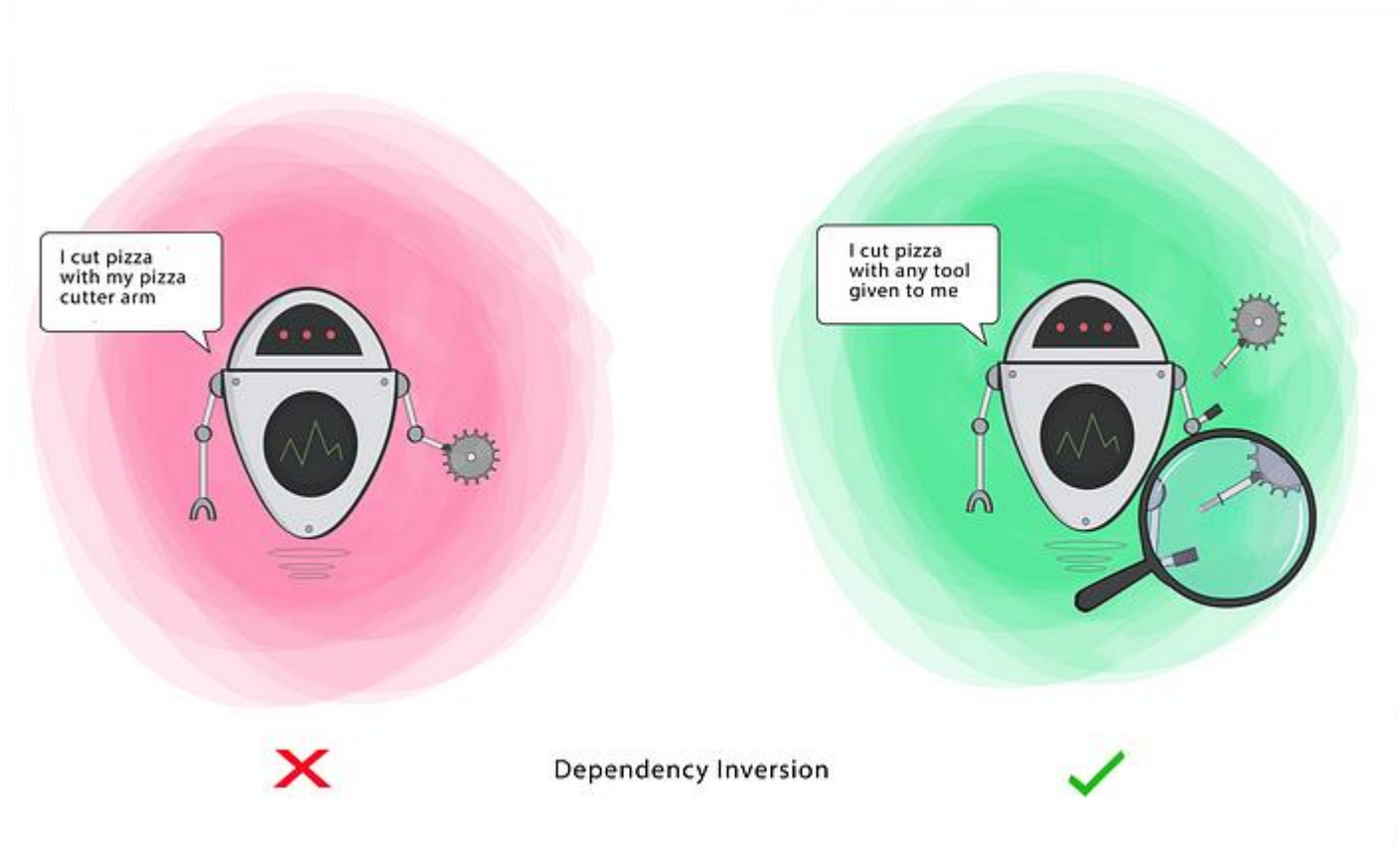
Quando uma classe é obrigada a executar ações que não são úteis, trata-se de um desperdício e pode produzir bugs inesperados se a classe não tiver a capacidade de executar essas ações. Uma classe deve executar apenas as ações necessárias para cumprir seu papel. Qualquer outra ação deve ser removida completamente ou movida para outro lugar se puder ser utilizada por outra classe no futuro. *Entendido?*

O objetivo desse princípio é dividir um conjunto de ações em conjuntos menores para que uma classe execute **apenas** o conjunto de ações de que necessita. Na imagem à esquerda, temos dois robôs: um com antena e outro sem antena. E todos os robôs possuem três ações: girar, rodar os braços e balançar as antenas. *Ora, se o segundo robô não possui antena, como ele conseguirá executar a terceira função?* Ele não conseguirá, logo essa ação é inútil!

Já na imagem à direita, também temos dois robôs: um com antena e outro sem antena. No entanto, temos uma divisão de ações: ações para robôs que podem girar; ações para robôs que podem rodar os braços; e ações para robôs que podem balançar as antenas. Logo, nenhum robô possui uma

função inútil que não podem executar e que são apenas um desperdício capaz de produzir bugs inesperados. *Bacana?*

Inversão de Dependência



MÓDULOS DE ALTO NÍVEL NÃO DEVEM DEPENDER DE MÓDULOS DE BAIXO NÍVEL: AMBOS DEVEM DEPENDER DA ABSTRAÇÃO.

AS ABSTRAÇÕES NÃO DEVEM DEPENDER DE DETALHES: DETALHES DEVEM DEPENDER DE ABSTRAÇÕES.

Em primeiro lugar, vamos definir os termos usados aqui de forma mais simples:

- **Módulo/Classe de Alto Nível:** classe que executa uma ação com uma ferramenta;
- **Módulo/Classe de Baixo Nível:** ferramenta necessária para executar uma ação;
- **Abstração:** representa uma interface que conecta as duas classes;
- **Detalhes:** como a ferramenta funciona .

Este princípio diz que uma classe não deve ser fundida com a ferramenta que usa para executar uma ação. Em vez disso, ele deve ser fundido à interface que permitirá que a ferramenta se conecte à classe. Também diz que tanto a classe quanto a interface não devem saber como a ferramenta

funciona. No entanto, a ferramenta precisa atender à especificação da interface. O objetivo desse princípio é reduzir a dependência de uma classe de alto nível na classe de baixo nível, introduzindo uma interface.

Na imagem à esquerda, temos um robô que possui um braço exclusivamente para cortar pizzas; já na imagem à direita, temos um robô que possui um braço que pode ser adaptado para diversas ações – inclusive cortar pizza. Note que eu posso adaptar uma outra ferramenta no braço do robô para que ele possa realizar outra ação. Logo, temos uma inversão: as classes de alto nível deixam de depender das classes de baixo nível e passam a depender apenas da abstração (interface).

Para finalizar, vamos falar um pouquinho sobre coesão e acoplamento. A implementação de qualquer classe deve ser coesa, isto é, toda classe deve implementar uma única funcionalidade ou serviço. Especificamente, todos os métodos e atributos de uma classe devem estar voltados para a implementação do mesmo serviço. Uma outra forma de explicar coesão é afirmando que toda classe deve ter uma única responsabilidade no sistema.

Dito de outra forma, deve existir um único motivo para modificar uma classe. Ok? Bem, a coesão tem algumas vantagens: facilita a implementação de uma classe, bem como o seu entendimento e manutenção; facilita a alocação de um único responsável por manter uma classe; e facilita o reuso e teste de uma classe, pois é mais simples reusar e testar uma classe coesa do que uma classe com várias responsabilidades.

Já o acoplamento é a força da conexão entre duas classes. Trata-se da medida em que as partes de um programa estão interconectadas entre si. Pode-se dizer que se refere à quantidade de dependência entre os elementos de um programa. Quanto menor o acoplamento, melhor será a reutilização de código e a manutenção do programa. Na tabela seguinte, veremos um conjunto de propriedades de projeto relacionadas a cada um dos princípios:

PRINCÍPIOS SOLID	DESCRIÇÃO	PROPRIEDADES
RESPONSABILIDADE ÚNICA	Uma classe deve ter um, e somente um, motivo para mudar.	Coesão
ABERTO/FECHADO	Objetos ou entidades devem estar abertos para extensão, mas fechados para modificação.	Extensibilidade
SUBSTITUIÇÃO DE LISKOV	Uma classe derivada deve ser substituível por sua classe base.	Extensibilidade
SEGREGAÇÃO DE INTERFACE	Uma classe não deve ser forçada a implementar interfaces e métodos que não irá utilizar.	Coesão
INVERSÃO DE DEPENDÊNCIA	Dependa de abstrações e, não, de implementações.	Acoplamento

QUESTÕES COMENTADAS

1. (CESPE / SERPRO – 2023) O princípio de segregação de interface do SOLID determina a responsividade, em que uma mesma interface atende a vários aparelhos, sem necessidade de modificações individuais.

Comentários:

O Princípio de Segregação de Interface (Interface Segregation Principle - ISP) do SOLID não trata de responsividade ou de interfaces para diferentes aparelhos. Esse princípio estabelece que uma classe não deve ser obrigada a implementar interfaces que não utiliza. Em outras palavras, é melhor ter interfaces mais específicas e menores que atendam necessidades específicas, em vez de uma única interface grande que force classes a implementar métodos desnecessários. Isso promove maior flexibilidade e modularidade no design de software.

Gabarito: Errado

2. (VUNESP / UFABC – 2023) Em desenvolvimento de software orientado a objetos, o princípio SOLID conhecido como Inversão de Dependência estabelece que
- a) apenas alterações potenciais na especificação do software devem ser capazes de afetar a especificação de uma classe.
 - b) classes devem ser abertas para extensões e fechadas para modificações.
 - c) se deve separar funções de diferentes naturezas em interfaces diferentes, sem construir uma interface única de propósito geral.
 - d) uma implementação de função que recebe referências para objetos de determinada classe deve ser capaz de lidar com objetos de suas subclasses sem conhecê-las.
 - e) classes devem depender de interfaces ou classes abstratas, em vez de classes concretas.

Comentários:

- (a) Errado. Esta descrição não se alinha com a Inversão de Dependência. Parece se referir a uma ideia de controle de alterações, mas não está relacionada a dependências entre classes.
- (b) Errado. Esta descrição se refere ao Princípio Aberto/Fechado (Open/Closed Principle), que sugere que as classes devem ser abertas para extensão, mas fechadas para modificações.

(c) Errado. Esta descrição se refere ao Princípio de Segregação de Interfaces (Interface Segregation Principle), que indica que uma classe não deve ser forçada a implementar interfaces que não usa.

(d) Errado. Esta descrição está relacionada ao Princípio de Substituição de Liskov (Liskov Substitution Principle), que indica que objetos de uma classe devem poder ser substituídos por objetos de suas subclasses sem alterar a funcionalidade desejada.

(e) Correto. O Princípio de Inversão de Dependência (Dependency Inversion Principle) estabelece que: módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações (interfaces ou classes abstratas); abstrações não devem depender de detalhes. Detalhes devem depender de abstrações; em outras palavras, classes devem depender de interfaces ou classes abstratas, em vez de classes concretas, para reduzir o acoplamento e aumentar a flexibilidade e escalabilidade do código.

Gabarito: Letra E

3. (FGV / Senado Federal – 2022) Os princípios de Orientação a Objetos e Design de Código são guiados pelos conceitos do acrônimo SOLID, em que cada letra descreve um princípio. Assinale a opção que indica o princípio que tem a preocupação com a falta de coesão e alto acoplamento:

- a) Princípio da Substituição de Liskov.
- b) Princípio da Segregação de Interface.
- c) Princípio Aberto-fechado.
- d) Princípio da Responsabilidade Única.
- e) Princípio da Inversão de Dependência.

Comentários:

Estamos em busca do princípio que se preocupa com a falta de coesão e com o alto acoplamento. A falta de coesão indica que uma classe tem mais de uma atribuição, quando deveria ter apenas uma; o alto acoplamento indica que uma classe depende bastante de outras classes. *Qual é o princípio que busca inibir a falta de coesão?* Responsabilidade Única e Segregação de Interface. *Qual é o princípio que busca inibir o alto acoplamento?* Inversão de Dependência. Logo, temos três respostas corretas e, por essa razão, a questão foi anulada.

Gabarito: Anulada

4. (CESPE / BANRISUL – 2022) Os princípios de programação orientada a objetos que correspondem aos princípios SOLID são: criador (creator), especialista na informação (information expert), controlador (controller), polimorfismo (polymorphism), fabricação pura (pure fabrication).

Comentários:

Os Princípios SOLID são:

- [S] – *Single Responsibility Principle* (Princípio da Responsabilidade Única)
- [O] – *Open-Closed Principle* (Princípio Aberto-Fechado)
- [L] – *Liskov Substitution Principle* (Princípio da Substituição de Liskov)
- [I] – *Interface Segregation Principle* (Princípio da Segregação da Interface)
- [D] – *Dependency Inversion Principle* (Princípio da Inversão da Dependência)

Gabarito: Errado

5. (AOCP / PRODEB – 2018) Com base no modelo SOLID utilizado como referência para padrões de projeto e princípios arquiteturais, um dos seus princípios denominados de LSP (*Liskov Substitution Principle*) diz respeito ao fato de que:

- a) uma classe deve ter apenas uma razão para mudar, sendo coesa.
- b) os objetos devem ser substituíveis com instâncias de seus tipos base, sem prejudicar o funcionamento do software.
- c) todo o processo de desenvolvimento de software deve ser baseado em abstrações, já que elas pouco mudam.
- d) deve-se utilizar o conceito de herança o máximo possível, estendendo para todo e qualquer atributo que possua alguma semelhança.
- e) os módulos devem ser enxutos tendo poucos comportamentos.

Comentários:

(a) Errado, isso é o Princípio de Responsabilidade Única; (b) Correto; (c) Errado, isso é o Princípio de Inversão de Dependência; (d) Errado, deve-se evitar herança o máximo possível para evitar o acoplamento e devemos priorizar a composição ou interface; (e) Errado, essa descrição não está relacionada ao LSP.

Gabarito: Letra B

6. (AOCP / PRODEB – 2018) Em relação aos padrões de projeto de software e princípios arquiteturais, em programação orientada a objetos, existe um princípio denominado de SOLID. Ele, por sua vez, é composto por 05 princípios de acordo com as suas iniciais, sendo eles:

- a) S (Single responsibility principle) – O (Openclosed principle) – L (Liskov substitution principle) – I (Interface segregation principle) e D (Dependency inversion principle).

b) S (Solid principle) – O (Open principle) – L (Library principle) – I (Integration principle) – D (Double principle).

c) S (Security closed principle) – O (Open extend principle) – L (Liskov include principle) – I (Interface duplication principle) – D (Duplicate structure principle).

d) S (Single closed principle) – O (Open-closed principle) – L (Library exclusive principle) – I (Integration case principle) – D (Dependency inversion principle).

e) S (Security basic principle) – O (Open extern principle) – L (Liskov include principle) – I (Interface duplication principle) – D (Duplicate segregation principle).

Comentários:

Ele é composto por 05 princípios de acordo com as suas iniciais, sendo eles:

[S] – *Single Responsibility Principle* (Princípio da Responsabilidade Única)

[O] – *Open-Closed Principle* (Princípio Aberto-Fechado)

[L] – *Liskov Substitution Principle* (Princípio da Substituição de Liskov)

[I] – *Interface Segregation Principle* (Princípio da Segregação da Interface)

[D] – *Dependency Inversion Principle* (Princípio da Inversão da Dependência)

Gabarito: Letra A

7. (CESPE / BANRISUL – 2022) O princípio da segregação de interface dos padrões SOLID define que uma classe deve possuir somente uma operação para ser executada.

Comentários:

Na verdade, a descrição trata do Princípio da Responsabilidade Única. O Princípio da Segregação de Interface define que uma classe não deve ser forçada a implementar interfaces e métodos que não irá utilizar.

Gabarito: Errado

8. (FUNDEP / UFJF – 2022) No contexto dos princípios SOLID, analise as afirmativas a seguir.

I. O princípio de inversão de dependência estabelece que uma classe deve depender de implementações abstratas e não concretas, sempre que possível.

II. O princípio aberto / fechado estabelece que uma classe deve estar fechada para extensões, mas aberta para modificações.

III. O princípio da responsabilidade única é uma aplicação da propriedade de coesão, por propor que toda classe deve ter uma única finalidade.

Está(ão) correta(s) a(s) afirmativa(s):

- a) I, apenas.
- b) II, apenas.
- c) III, apenas.
- d) I e III, apenas.
- e) II e III, apenas.

Comentários:

(I) Correto; (II) Errado, ele estabelece que uma classe deve estar aberta para extensões, mas fechada para modificações – a questão inverteu os conceitos; (III) Correto.

Gabarito: Letra D

9. (FCC / TRE-PR – 2017) Os princípios SOLID reúnem cinco boas práticas para projetos Orientados a Objetos-OO. O princípio S, que se refere ao Single Responsibility Principle-SRP ou Princípio de Responsabilidade Única, indica que uma classe deve ter uma e, apenas uma, razão para mudar. Considere a classe Java abaixo.

```
public class UrnaEleitoral {  
    public void AdicionarCandidato(String nome, int numero, int partido) { }  
    public decimal CalcularTotalVotosCandidato() { }  
    public void CadastrarPartidos() { }  
    public void CadastrarEleitores() { }  
    public void CadastrarMesarios() { }  
}
```

Com base no princípio SRP e nas boas práticas para projetos OO, é correto afirmar:

- a) O SRP visa aumentar o acoplamento entre classes e separar responsabilidades como forma de melhorar o código da aplicação OO sendo desenvolvida.
- b) A classe UrnaEleitoral tem acoplamento baixo, ou seja, tem um número pequeno de dependências e, portanto, fica mais sujeita a mudanças em decorrência de alterações em outras classes.
- c) Uma classe com mais de um motivo para mudar possui mais de uma responsabilidade e apresentando dificuldade de manutenção, mas, por outro lado, tem maior facilidade de reuso e de coesão.
- d) A classe UrnaEleitoral apresenta uma quebra do SRP, uma vez que possui responsabilidades que deveriam ser de componentes distintos do software.

e) Em um projeto com várias classes seguindo o padrão da classe UrnaEleitoral fica mais fácil manter a coesão em um nível mais alto ou em nível de componentes, pois o software fica com uma divisão clara de camadas.

Comentários:

(a) Errado, ele visa reduzir o acoplamento entre classes; (b) Errado, é possível notar que há vários métodos relativos a domínios diferentes de urna, tais como Candidato, Partidos, Eleitores e Mesários. Logo, ela tem muitas dependências, portanto alto acoplamento; (c) Errado, se ela possui mais de uma responsabilidade, ela tem menos facilidade de reúso e menor coesão; (d) Correto, note que ela tem cinco métodos, portanto tem cinco responsabilidades diferentes – quando deveria ter apenas uma; (e) Errado, fica mais difícil de manter visto que ela tem diversas responsabilidades.

Gabarito: Letra D

10. (IESES / CREA-SC – 2017) Assinale a alternativa correta:

- a) SOLID é um acróstico e, cada letra está relacionada a um princípio para programação e design orientado a objetos de autoria de Robert C. Martin. O Acróstico é formado pela inicial de Sistema, Objeto, Lógica, Informação e, Design.
- b) SOLID é um acróstico e, cada letra está relacionada a um princípio para programação e design orientado a objetos de autoria de Robert C. Martin.
- c) SOLID é um acróstico formado pelas iniciais de SPR, OCP, LSP, ISP e DIP. É um conjunto consistente de princípios para modelagem matemática e computacional de sólidos tridimensionais. A modelagem sólida distingue-se das áreas relacionadas de modelagem geométrica e computação gráfica por sua ênfase na fidelidade física.
- d) Em programação Orientada a Objetos é um conjunto consistente de princípios para modelagem matemática e computacional de sólidos tridimensionais. A modelagem sólida distingue-se das áreas relacionadas de modelagem geométrica e computação gráfica por sua ênfase na fidelidade física.

Comentários:

Em primeiro lugar, um acróstico é uma palavra formada pelas primeiras letras de cada linha conforme podemos ver a seguir:

- [S] – *Single Responsibility Principle* (Princípio da Responsabilidade Única)
- [O] – *Open-Closed Principle* (Princípio Aberto-Fechado)
- [L] – *Liskov Substitution Principle* (Princípio da Substituição de Liskov)

[I] – *Interface Segregation Principle* (Princípio da Segregação da Interface)

[D] – *Dependency Inversion Principle* (Princípio da Inversão da Dependência)

(a) Errado, ele é formado pela inicial de Single, Open, Liskov, Interface e Dependency; (b) Correto; (c) Errado, não tem nenhuma relação com modelagem matemática e computacional de sólidos tridimensionais; (d) Errado, não tem nenhuma relação com modelagem matemática e computacional de sólidos tridimensionais.

Gabarito: Letra B

LISTA DE QUESTÕES

1. **(CESPE / SERPRO – 2023)** O princípio de segregação de interface do SOLID determina a responsividade, em que uma mesma interface atende a vários aparelhos, sem necessidade de modificações individuais.
2. **(VUNESP / UFABC – 2023)** Em desenvolvimento de software orientado a objetos, o princípio SOLID conhecido como Inversão de Dependência estabelece que
 - a) apenas alterações potenciais na especificação do software devem ser capazes de afetar a especificação de uma classe.
 - b) classes devem ser abertas para extensões e fechadas para modificações.
 - c) se deve separar funções de diferentes naturezas em interfaces diferentes, sem construir uma interface única de propósito geral.
 - d) uma implementação de função que recebe referências para objetos de determinada classe deve ser capaz de lidar com objetos de suas subclasses sem conhecê-las.
 - e) classes devem depender de interfaces ou classes abstratas, em vez de classes concretas.
3. **(FGV / Senado Federal – 2022)** Os princípios de Orientação a Objetos e Design de Código são guiados pelos conceitos do acrônimo SOLID, em que cada letra descreve um princípio. Assinale a opção que indica o princípio que tem a preocupação com a falta de coesão e alto acoplamento:
 - a) Princípio da Substituição de Liskov.
 - b) Princípio da Segregação de Interface.
 - c) Princípio Aberto-fechado.
 - d) Princípio da Responsabilidade Única.
 - e) Princípio da Inversão de Dependência.
4. **(CESPE / BANRISUL – 2022)** Os princípios de programação orientada a objetos que correspondem aos princípios SOLID são: criador (creator), especialista na informação (information expert), controlador (controller), polimorfismo (polymorphism), fabricação pura (pure fabrication).
5. **(AOCP / PRODEB – 2018)** Com base no modelo SOLID utilizado como referência para padrões de projeto e princípios arquiteturais, um dos seus princípios denominados de LSP (*Liskov Substitution Principle*) diz respeito ao fato de que:

- a) uma classe deve ter apenas uma razão para mudar, sendo coesa.
- b) os objetos devem ser substituíveis com instâncias de seus tipos base, sem prejudicar o funcionamento do software.
- c) todo o processo de desenvolvimento de software deve ser baseado em abstrações, já que elas pouco mudam.
- d) deve-se utilizar o conceito de herança o máximo possível, estendendo para todo e qualquer atributo que possua alguma semelhança.
- e) os módulos devem ser enxutos tendo poucos comportamentos.

6. (AOCP / PRODEB – 2018) Em relação aos padrões de projeto de software e princípios arquiteturais, em programação orientada a objetos, existe um princípio denominado de SOLID. Ele, por sua vez, é composto por 05 princípios de acordo com as suas iniciais, sendo eles:

- a) S (Single responsibility principle) – O (Open/closed principle) – L (Liskov substitution principle) – I (Interface segregation principle) e D (Dependency inversion principle).
- b) S (Solid principle) – O (Open principle) – L (Library principle) – I (Integration principle) – D (Double principle).
- c) S (Security closed principle) – O (Open extend principle) – L (Liskov include principle) – I (Interface duplication principle) – D (Duplicate structure principle).
- d) S (Single closed principle) – O (Open-closed principle) – L (Library exclusive principle) – I (Integration case principle) – D (Dependency inversion principle).
- e) S (Security basic principle) – O (Open extern principle) – L (Liskov include principle) – I (Interface duplication principle) – D (Duplicate segregation principle).

7. (CESPE / BANRISUL – 2022) O princípio da segregação de interface dos padrões SOLID define que uma classe deve possuir somente uma operação para ser executada.

8. (FUNDEP / UFJF – 2022) No contexto dos princípios SOLID, analise as afirmativas a seguir.

- I. O princípio de inversão de dependência estabelece que uma classe deve depender de implementações abstratas e não concretas, sempre que possível.
- II. O princípio aberto / fechado estabelece que uma classe deve estar fechada para extensões, mas aberta para modificações.
- III. O princípio da responsabilidade única é uma aplicação da propriedade de coesão, por propor que toda classe deve ter uma única finalidade.

Está(ão) correta(s) a(s) afirmativa(s):

- a) I, apenas.

- b) II, apenas.
- c) III, apenas.
- d) I e III, apenas.
- e) II e III, apenas.

9. (FCC / TRE-PR – 2017) Os princípios SOLID reúnem cinco boas práticas para projetos Orientados a Objetos-OO. O princípio S, que se refere ao Single Responsibility Principle-SRP ou Princípio de Responsabilidade Única, indica que uma classe deve ter uma e, apenas uma, razão para mudar. Considere a classe Java abaixo.

```
public class UrnaEleitoral {  
    public void AdicionarCandidato(String nome, int numero, int partido) { }  
    public decimal CalcularTotalVotosCandidato() { }  
    public void CadastrarPartidos() { }  
    public void CadastrarEleitores() { }  
    public void CadastrarMesarios() { }  
}
```

Com base no princípio SRP e nas boas práticas para projetos OO, é correto afirmar:

- a) O SRP visa aumentar o acoplamento entre classes e separar responsabilidades como forma de melhorar o código da aplicação OO sendo desenvolvida.
- b) A classe UrnaEleitoral tem acoplamento baixo, ou seja, tem um número pequeno de dependências e, portanto, fica mais sujeita a mudanças em decorrência de alterações em outras classes.
- c) Uma classe com mais de um motivo para mudar possui mais de uma responsabilidade e apresentando dificuldade de manutenção, mas, por outro lado, tem maior facilidade de reuso e de coesão.
- d) A classe UrnaEleitoral apresenta uma quebra do SRP, uma vez que possui responsabilidades que deveriam ser de componentes distintos do software.
- e) Em um projeto com várias classes seguindo o padrão da classe UrnaEleitoral fica mais fácil manter a coesão em um nível mais alto ou em nível de componentes, pois o software fica com uma divisão clara de camadas.

10. (IESES / CREA-SC – 2017) Assinale a alternativa correta:

- a) SOLID é um acróstico e, cada letra está relacionada a um princípio para programação e design orientado a objetos de autoria de Robert C. Martin. O Acróstico é formado pela inicial de Sistema, Objeto, Lógica, Informação e, Design.

b) SOLID é um acróstico e, cada letra está relacionada a um princípio para programação e design orientado a objetos de autoria de Robert C. Martin.

c) SOLID é um acróstico formado pelas iniciais de SPR, OCP, LSP, ISP e DIP. É um conjunto consistente de princípios para modelagem matemática e computacional de sólidos tridimensionais. A modelagem sólida distingue-se das áreas relacionadas de modelagem geométrica e computação gráfica por sua ênfase na fidelidade física.

d) Em programação Orientada a Objetos é um conjunto consistente de princípios para modelagem matemática e computacional de sólidos tridimensionais. A modelagem sólida distingue-se das áreas relacionadas de modelagem geométrica e computação gráfica por sua ênfase na fidelidade física.

GABARITO

1. ERRADO
2. LETRA E
3. ANULADA
4. ERRADO
5. LETRA B
6. LETRA A
7. ERRADO
8. LETRA D
9. LETRA D
10. LETRA B

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.